

# How to store passwords in your database

Explained as simply as possible... but not simpler.



STEPHANE MOREAU

NOV 10, 2025



46



1



3

Share

It is essential to store passwords in a way that prevents an attacker from getting them, even if the database is compromised.

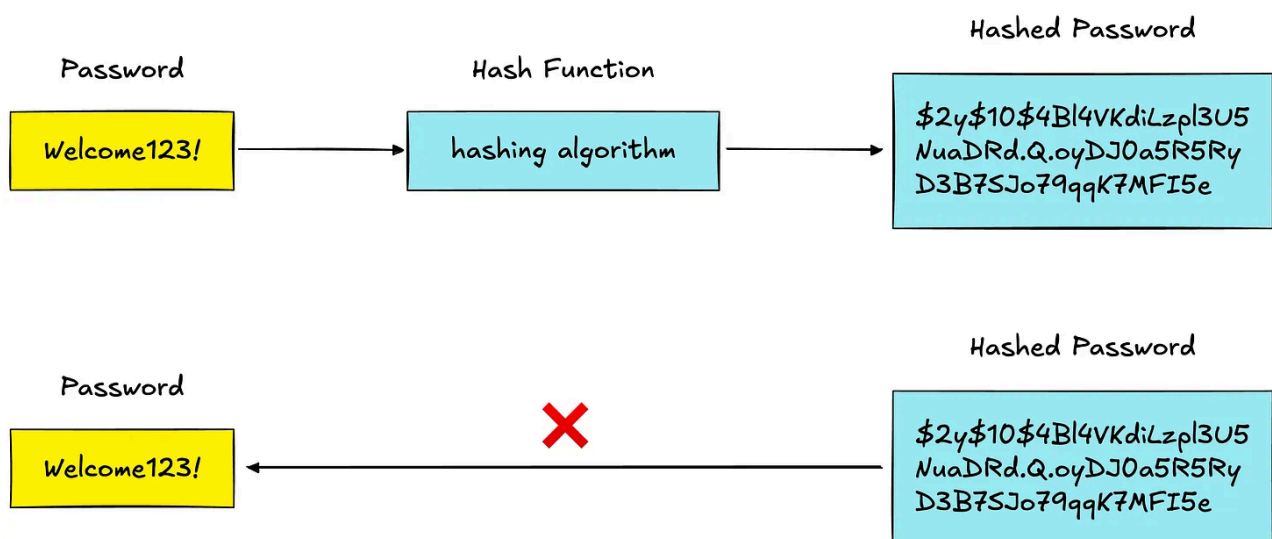
Never store passwords as plaintext.

## Recommendations from OWASP

According to the [Open Web Application Security Project \(OWASP\)](#), secure password storage requires two key ingredients:

### 1. Use a Modern Hashing Algorithm

A hash function is a one-way mathematical operation. You can turn a password into a hash, but you cannot reverse a hash back into the password.



But not all hashing functions are equal.

Common hash functions like MD5 and SHA-1 are fast - which sounds good, but is actually terrible for passwords. Fast hashing means attackers can try billions of

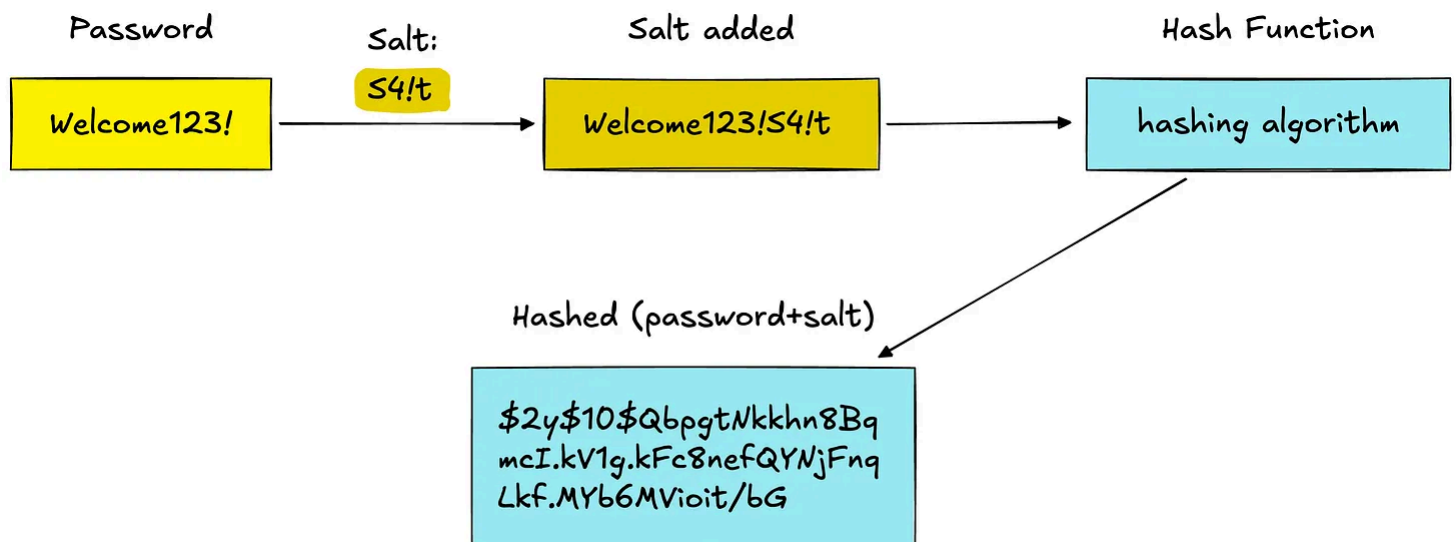
password guesses per second using brute-force attacks.

Instead, use password-specific hashing algorithms that are intentionally slow and resource-intensive (such as [bcrypt](#)).

These algorithms make brute-force attacks computationally expensive and unattractive.

## 2. Salt the Passwords

A salt is a unique, randomly generated string added to each password before hashing. Even if two users have the same password ("password123"), their hashes will be completely different.



Why is this important?

Without salts, attackers can use precomputed attack methods:

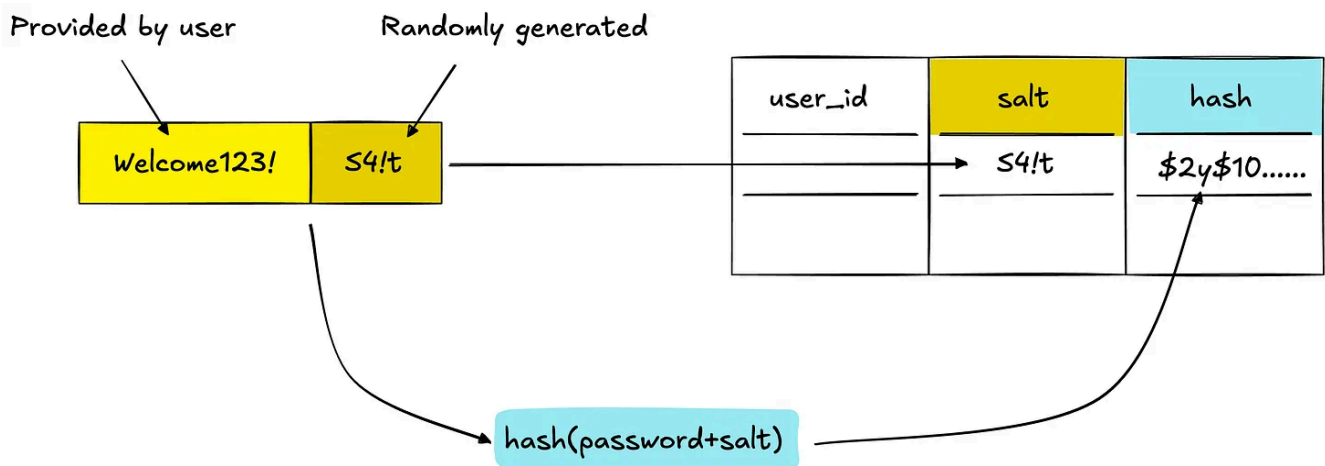
- **Rainbow tables:** Massive databases of precomputed hashes for common passwords
- **Database lookups:** Quick searches to crack passwords in seconds

By adding a unique salt to each password, you ensure every hash is unique, making these precomputed attacks useless.

## How It Works: Registration Flow

When a user creates an account:

1. User submits their password
2. System generates a random salt (e.g., “k8Px2mN9”)
3. System combines password + salt
4. System computes hash using a hashing algorithm like bcrypt
5. System stores both the hash AND the salt in the database



The salt isn't a secret. It can be stored as plaintext right next to the hash. Its job is to make each hash unique, not to hide information.

**Example:**

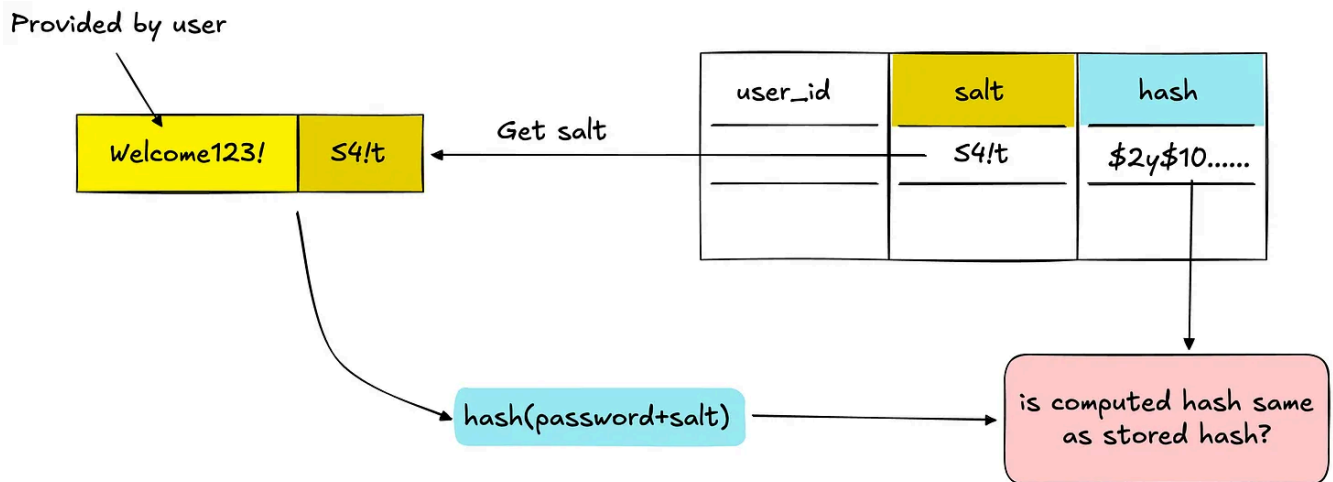
```
Password: "Welcome123!"  
Salt: "S4!t"  
Combined: "Welcome123!S4!t"  
Hash: "$2y$10$Qbpn8Bq...b6MVioit/bG" (this gets stored)
```

## How It Works: Login Validation

When a user logs in:

1. User submits username and password
2. System fetches the stored hash and salt from the database
3. System appends the stored salt to the submitted password

4. System computes a new hash
5. System compares: if computed hash == stored hash, password is valid ✓



## Additional Security Layers

Consider these complementary defenses:

- **Rate limiting:** Limit login attempts per IP address or user account
- **Account lockout:** Temporarily disable accounts after multiple failed attempts
- **Adaptive hash parameters:** Increase bcrypt “cost factor” over time as hardware improves
- **Pepper (optional):** Add a secret key stored outside the database for an extra layer

## Like posts like this?

By subscribing, you get a breakdown like this one every week.

Free subscribers also get a little bonus:

 **The System Design Interview Preparation Cheat Sheet**

If you're into visuals, paid subscribers unlock:

- My Excalidraw system design template – so you have somewhere to start
- My Excalidraw component library – used in the diagram of this issue

No pressure though. Your support helps me keep writing, and I appreciate it more than you know ❤️

System design, explained ridiculously simply.  
Finally, make sense of system design and pass  
your next interview!

Subscribe

Discussion about this post

Comments

Restacks



Write a comment...



Daniil Shykhov 10 de nov. de 2025



♥️ Liked by Stephane Moreau

Clean, simple, and practical, this is how security concepts should be taught.

♥️ LIKE (3)    💬 REPLY

🔗 SHARE